**Institut catholique des arts et métiers**
**Université catholique d'Afrique centrale - Université Loyola du Congo**

# Théorie et modèles relationnels

*Fondements*

**TMR_02**

Christina KHNAISSER *(christina.khnaisser@usherbrooke.ca)*

Luc LAVOIE *(luc.lavoie@usherbrooke.ca)*

*Scriptorum/Scriptorum/TMR_02-Fondements, version 0.1.5.c, en date du 2025-04-14*

*— version applicable —*

© 2026, CoFELI [CC BY-NC-SA 4.0]

# Plan

Introduction . . . . . 3
1. Présentation . . . . . 4
2. Théorie relationnelle. . . . . 26
3. Opérateurs relationnels . . . . . 54

© 2026, CoFELI [CC BY-NC-SA 4.0]

# Introduction

Le présent document a pour but de présenter une synthèse de la théorie relationnelle proposée par Edgar F. Codd et développée grâce, notamment, aux contributions de Christopher J. Date, Raymond F. Boyce, Nikos A. Lorentzos et Jeffrey D. Ullman.

La présentation repose sur des bases minimales :

- la logique du premier ordre,
- la théorie des ensembles (dont la théorie des types),
- l'arithmétique entière,
- les langages rationnels.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1. Présentation

La logique et la déduction rationnelle sont les outils privilégiés par la science pour encadrer notre perception du monde, le penser et l'expliquer.

Plus récemment, Floyd, Codd, Hoare, Dijkstra, Gries, Date, Ullman et d'autres ont contribué à rendre effectivement calculable un sous-ensemble significatif des propriétés des systèmes construits sur les bases établies par leurs prédécesseurs.

Ces contributions sont le fondement des systèmes de gestion de bases de données (SGBD) les plus performants développés à ce jour, à savoir les SGBD relationnels (SGBDR).

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1.1. Intuitions à l’origine de la théorie relationnelle

- Le point de départ : l’équivalence entre fait, proposition et tuple.
- Une généralisation : l’équivalence entre catégorie, prédicat et relation.
- Le couplage de la théorie relationnelle à la théorie des types.
- L’automatisation du calcul.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1.2. Pourquoi le modèle relationnel ?

Lorsque Codd jette les bases de la théorie relationnelle en 1969 et propose un premier modèle opérationnalisable en 1970, il s'inscrit en continuité d'une histoire déjà longue dont voici quelques jalons choisis :

- Les fonctionnaires babyloniens (XVIII$^{e}$ siècle avant notre ère).
- Ératosthène (III$^{e}$ siècle avant notre ère) ;
- Les ingénieurs romains (I$^{er}$ siècle) ;
- Cardan et Kepler (XVI$^{e}$ siècle) ;

© 2026, CoFELI [CC BY-NC-SA 4.0]

Ce n’est toutefois pas cette histoire qui permit à la théorie et au modèle de Codd de s’imposer rapidement à partir des années 1970 et de rester dominant jusqu’à nos jours, mais

- la capacité de définir des modèles de données tant simples que très complexes ;
- la capacité de les opérationnaliser sur la base de raisonnements fondés sur la logique du premier ordre dont le calcul est totalement automatisé ;
- la capacité d’évaluer objectivement plusieurs critères d’adéquation d’une solution ainsi formulée en regard du problème initialement posé ;

© 2026, CoFELI [CC BY-NC-SA 4.0]

notamment grâce à

- l'utilisation conjointe des notations ensemblistes, relationnelles et logiques ;
- l'utilisation d'attributs dénotationnels plutôt que positionnels ;
- l'intégration des notions d'équivalence et d'égalité ;
- la complétude calculatoire au sens de Turing qui en résulta.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Adéquation*

Des huit principaux critères d'adéquation (voir TMR_01), sept sont couverts de façon utile, voire complète, par la théorie relationnelle, ses bases et les théories informatiques complémentaires (telle que la théorie de la complexité). Le huitième critère, l'éthique (la conformité aux principes et règles de conduite d'une société humaine de référence), y échappe cependant.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Modèles et méta-modèles*

Les modèles issus de la théorie relationnelle permettent la définition de modèles logiques de données. Ce sont donc ce qu'il est convenu d'appeler des méta-modèles. De plus, la théorie relationnelle a la particularité de pouvoir se décrire elle-même et il peut en être de même des modèles qui la représentent ! Cette propriété incite donc la plupart des concepteurs de SBGB à y inclure une base de données, appelée catalogue, décrivant les bases de données gérées par le SGBD (dont le catalogue lui-même). Cette approche a l'avantage de permettre l'utilisation des opérateurs relationnels (et de tout l'environnement d'exploitation) lors de la gestion et de l'évolution des bases de données du SGBD.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1.3. Théorie des ensembles

## Ensemble

Un ensemble est la réunion d'éléments distincts qui peut être comprise comme un tout [Cantor 1895].

*Exemple 1. Ensembles*

1. *{1, 20, 80, 46, 50}*
2. *{Paul, Éliane, Mohamed, Sergueï, Diego}*
3. *{Blanc-Sablon, ᐳᐃᑦᓂᑐᑦᑲ, Tadoussac, Chandler, San Diego}*
4. *{IFT187, IGE687, IFT159, IFT677, STT418}*

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Tableau 1. Principaux opérateurs ensemblistes*

| Symbole | Spécification |
|---|---|
| $\in$ | Appartenance |
| $\subset$ | Inclusion |
| $=$ | Égalité |
| $\cup$ | Union |
| $\cap$ | Intersection |
| $\setminus$ | Différence |
| $\times$ | Produit cartésien |

Une présentation plus complète est disponibles dans les notes de cours.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## 1.3.1. Les ensembles au sein de la théorie relationnelle

Dans le cadre de la théorie relationnelle, les éléments d'un ensemble sont toujours contraints par un type associé à l'ensemble lui-même. Ce type est désigné comme le « type de référence » de l'ensemble et noté *typeref(e)* où *e* est un ensemble.

Cette association peut être explicite (par voie de déclaration) ou déduite grâce au contexte.

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Raccourcis*

Dans la mesure où tous les ensembles ont un type de référence, l'opérateur « \ » permet de calculer le complément d'un sous-ensemble par rapport à son type de référence.

**\ e ≜ typeref(e) – e**

où *e* est une expression dont la valeur est un ensemble typé.

*Notes*

- En Tutorial D, cet opérateur n'est utilisable que dans un contexte de dénotation explicite des valeurs, ainsi `{ALL BUT e₁, …, eₙ}` dénote $\backslash\{ e_1, \ldots, e_n \}$. On aurait souhaité, la syntaxe plus générale suivante `ALL BUT exp`, où `exp` dénote toute expression ensembliste typée.
- En Discipulus, la syntaxe générale est utilisée et les expressions ensemblistes (en particulier d'ensembles d'identifiants d'attributs) sont dénotables.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## 1.3.2. Le couple

Le couple (la paire) est la formation d'une nouvelle entité (ou élément) à partir de deux autres.

*Dénotation des valeurs et propriétés*

**(x ; y)**

pour x et y des expressions dénotant des valeurs appropriées.

*Opérateurs usuels*

**@1(x ; y) = x**

l'élément gauche du couple ;

**@2(x ; y) = y**

l'élément droit du couple.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Équivalence*

- Il est possible de reformuler le couple en termes d'ensemble, ainsi $(x; y)$ est équivalent à l'ensemble ordonné $\{\{1, \{x\}\}, \{2, \{y\}\}\}$.
- L'utilisation du couple en tant que structure propre relève donc uniquement de la commodité.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1.4. Logique des propositions

## Proposition

Une proposition est une assertion ayant une valeur de vérité (vrai ou faux) indépendamment du temps, du lieu ou de la personne qui le prononce.

*Tableau 2. Opérateurs de la logique des propositions*

| Symbole | Spécification |
|---|---|
| $\neg$ | Négation (non) |
| $\wedge$ | Conjonction (et) |
| $\vee$ | Disjonction (ou) |
| $\Rightarrow$ | Implication (si…alors) |
| $\Leftrightarrow$ | Équivalence (si et seulement si) |
| $\perp$ | Faux |

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Exemple 2. Propositions*

- La note de `80` équivaut à la côte `A`.
- L'activité `IFT187` est préalable à l'activité `IGE487`.
- `Éliane` habite à `Blanc-Sablon`.
- `Éliane` n'est pas inscrite à `IFT187`.
- `Éliane` est inscrite à au moins quatre activités.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1.5. Logique des prédicats

La logique des prédicats généralise la logique des propositions. Elle s'intéresse à la formalisation de la sémantique des propositions.

**Prédicat**

> Un prédicat est une expression dont la valeur de vérité dépend des valeurs des variables dont telle est composée.

Nous utiliserons les opérateurs suivants pour les expressions logiques

(adaptation libre de https://fr.wikipedia.org/wiki/Portail:Logique, consulté le 2024-04-07).

© 2026, CoFELI [CC BY-NC-SA 4.0]

| Symbole | Spécification |
|---|---|
| $\neg A$ | Négation de A. Non A. |
| $A \wedge B$ | Conjonction. A Et B. |
| $A \vee B$ | Disjonction (inclusive). A Ou B. |
| $A \Rightarrow B$ | Implication. Si A alors B. |
| $A \Leftrightarrow B$ | Équivalence. A est équivalent à B ; on dit aussi : A si et seulement si B. |
| $\Gamma \vdash A$ | Déduction. De l'ensemble de formules Γ on déduit A. |
| $M \Vdash A$ | Réalisabilité. M réalise A, on dit aussi que M « force » A. |
| $M \vDash A$ | Modélisation. M est un modèle de A ; on dit aussi A est vraie dans M. |
| $\vdash A$ | Théorème. Notion syntaxique. |
| $\vDash A$ | Tautologie. Notion sémantique. |
| $(\forall v)\, A$ | Pour chaque v, A est vrai |
| $(\exists v)\, A$ | Il existe un v où A est vrai |

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Exemple 3. Prédicats*

- La personne étudiante `e` est inscrit à l'activité `a`.
- L'activité `a` est sous la responsabilité du département `d`.
- L'activité `a` est enseignée par la personne `p`.
- L'activité `a` est préalable à l'activité `b`.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 1.6. Relation mathématique

En mathématique, une relation entre deux ensembles est un sous-ensemble du produit cartésien de ces deux ensembles.

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Exemple*

Soit les deux ensembles suivants :

*Nom* = {*Paul, Éliane, Mohamed, Sergueï, Diego, Eurydice*}
*Ville* = {*Blanc-Sablon, ᐳᓇᕐᓂᑐᕐᑲ, Tadoussac, Chandler, San Diego, Sherbrooke*}

Le produit cartésien *Nom* × *Ville* produit 36 couples de valeurs (chaque nom avec chaque ville). Les ensembles suivants sont des relations entre Nom et Ville :

1. {(*Paul, ᐳᓇᕐᓂᑐᕐᑲ*), (*Éliane, Blanc-Sablon*), (*Mohamed, Tadoussac*), (*Sergueï, Chandler*)}
2. {(*Paul, Blanc-Sablon*), (*Mohamed, Chandler*)}
3. {}

© 2026, CoFELI [CC BY-NC-SA 4.0]

Les ensembles suivants ne sont pas des relations entre Nom et Ville :

4. *{(Arthur, Blanc-Sablon), (Mohamed, Montréal)}*
   Selon les définitions, Arthur n'est pas un Nom, Montréal n'est pas une ville.
5. *{(ᐳᕕᕐᓂᑐᖅ, Paul)}*
   Selon les définitions, ᐳᕕᕐᓂᑐᖅ est une Ville, Paul est un Nom, mais l'ordre couple n'est pas respecté.
6. *{(Paul, Blanc-Sablon), (Paul, Blanc-Sablon)}*
   Deux valeurs identiques ne peuvent apparaitre dans un même ensemble.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Notation*

Une relation peut également être représentée sous forme tabulaire, ainsi la relation 1 peut être représentée comme suit

| | |
|---|---|
| Paul | >ᕕᕐᓂᑐᖅ |
| Éliane | Blanc-Sablon |
| Mohamed | Tadoussac |
| Sergeï | Chandler |

## *Généralisation*

Par extension, une relation de degré $n$ est définie comme un sous-ensemble du produit cartésien de $n$ ensembles. Les couples (doublets) deviennent alors des triplets, des quadruplets… des multiplets.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 2. Théorie relationnelle

## 2.1. Définition informelle

### *Représentations de la relation informatique*

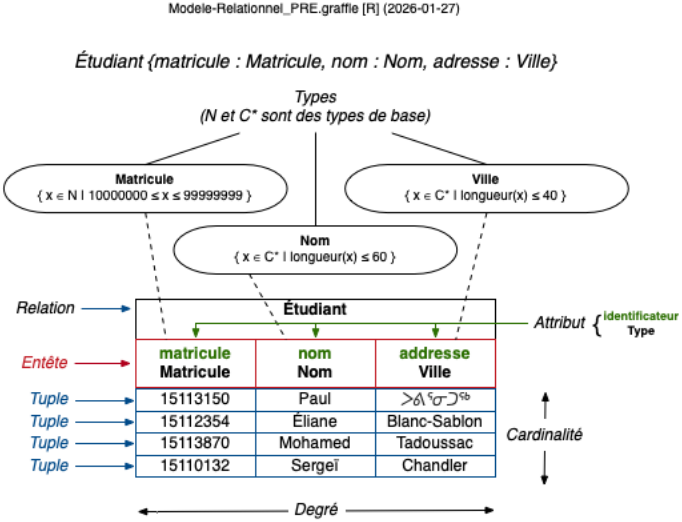
D’abord, voici la représentation usuelle, simple et pratique d’une relation informatique :

*Tableau 3. Étudiant*

| matricule | nom | ville |
|---|---|---|
| 15113150 | Paul | ᐳᕕᕐᓂᑐᖅ |
| 15112354 | Éliane | Blanc-Sablon |
| 15113870 | Mohamed | Tadoussac |
| 15110132 | Sergeï | Chandler |

© 2026, CoFELI [CC BY-NC-SA 4.0]

# Voici une seconde représentation, plus élaborée et plus précise :




© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Relation et prédicat*

La relation informatique permet de structurer, d'organiser, les données. La relation nous permettra bientôt de définir différents opérateurs pour interroger, manipuler et traiter ces données. Toutefois, pour les interpréter, il faut associer un prédicat à la relation afin de permettre de déterminer la proposition exacte correspondant à chaque tuple.

Voir l'exemple ci-après.

© 2026, CoFELI [CC BY-NC-SA 4.0]

**Si le prédicat suivant est associé à la relation *Étudiant***

*« L'étudiant identifié par le matricule "matricule" s'appelle "nom" et est né à "adresse" »,*

**alors la proposition correspondant au tuple**

*[matricule:15113150, nom:Paul, adresse:ᐳᕕᕐᓂᑐᖅ]*

**est**

*« L'étudiant identifié par le matricule "15113150" s'appelle "Paul" et est né à "ᐳᕕᕐᓂᑐᖅ" ».*

**Alors que, si c'est le prédicat**

*« L'étudiant identifié par le matricule "matricule" est en stage à "adresse" sous la direction de "nom" »,*

**la proposition correspondante est plutôt**

*« L'étudiant identifié par le matricule "15113150" est en stage à "ᐳᕕᕐᓂᑐᖅ" sous la direction de "Paul" ».*

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Remarques*

- Le prédicat ne peut être déduit de la relation.
- La forme du prédicat dépend de la relation.
- Lorsqu'un prédicat est bien formé, il y a une bijection entre ses variables et les attributs de la relation qu'il documente.
- Toute relation doit être documentée explicitement, sans quoi l'interprétation (le sens) des tuples qu'elle contient ne peut être établie.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Théorie relationnelle et modèles relationnels*

La théorie relationnelle est construite sur la base d'un système logique et de la théorie des ensembles qui permettent de définir les relations, les tuples, les attributs et leurs opérateurs. Afin de permettre la vérification et le traitement des valeurs associées aux attributs, la théorie relationnelle s'appuie également sur d'autres modèles fondamentaux, tels que l'arithmétique entière et les langages rationnels.

Plusieurs modèles relationnels peuvent alors être définis selon le modèle logique choisi, les types de bases retenus et le modèle de typage permettant de définir d'autres types s'ajoutant aux types de base.

Ainsi, au type regroupant les valeurs requises par le système logique considéré, les entiers, les textes et quelques autres types seront définis comme types de base.

© 2026, CoFELI [CC BY-NC-SA 4.0]

## 2.2. Type

Si la théorie relationnelle peut s'accommoder de plusieurs modèles de typage, certaines exigences doivent être respectées.

- Malheureusement, il n'y a pas de consensus relativement à ces exigences (notamment, celles du comité ISO 9075 définissant la norme SQL diffèrent de celles énoncées par Codd et développées par Date et Darwen).
- Nous nous en tiendrons donc ici à un modèle de typage très simple, également acceptable en regard des jeux d'exigences les plus courants.

© 2026, CoFELI [CC BY-NC-SA 4.0]

Une donnée est une valeur associée à deux représentations :

- l'une interne, apte à être acquise, stockée, transformée et retransmise par un dispositif physique, tel qu'un ordinateur ;
- l'autre externe, apte à être utilisée comme élément du vocabulaire d'un langage formel.

© 2026, CoFELI [CC BY-NC-SA 4.0]

Dans le cadre de la théorie minimaliste de typage utilisée dans le présent document, un type est soit un type de base, soit un sous-type.

- Un type de base (appelé aussi type racine ou domaine) est un ensemble fini de valeurs propres (c'est-à-dire que les valeurs d'un type de base n'appartiennent à aucun autre type de base). Une valeur est donc un élément d'un type de base.
- Un sous-type (appelé aussi type dérivé) est un sous-ensemble d'un type, sous-ensemble déterminé par une contrainte explicite (qui restreint les valeurs acceptées dans le sous-ensemble).

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Exemples*

- Un type de base prédéfini
  - Rel : *Entier*
  - SQL : `Integer`
- Une définition de type de base
  - Rel : *type Enseigne enum (coeur, carreau, trèfle, pique) ;*
  - SQL : `create type Enseigne as enum ('coeur','carreau','trèfle','pique');`
- Une définition de sous-type
  - Rel : *type Note : Entier | (0 ≤ valeur ≤ 100)*
  - SQL : `create domain Note as Integer check (value between 0 and 100) ;`

© 2026, CoFELI [CC BY-NC-SA 4.0]

- Une ellipse
  - Rel : *type Ellipse tuple (x1, y1, x2, y2, r : Nombre) ;*
  - SQL : `create type Ellipse (x1 Numeric, y1 Numeric, x2 Numeric, y2 Numeric, r Numeric);`
- Un cercle
  - Rel : *type Cercle : Ellipse | (x1=x2 ∧ x2=y2) ;*
  - SQL : `create domain Ellipse as Cercle check (x1=x2 and y1=y2) ;`
- Remarquer que le Cercle est logiquement un sous-type d'Ellipse, comme en mathématiques.
- Il ne faut donc pas confondre le concept de sous-typage (sémantique) et le concept d'héritage (syntaxique).

© 2026, CoFELI [CC BY-NC-SA 4.0]

## 2.3. Attribut

Un attribut est un couple formé d'un identifiant $a$ et d'un type $D$, noté $a{:}D$.

Par abus de langage, lorsque le contexte le permet, il est usuel de désigner l'attribut par son seul identifiant ; ainsi peut-on écrire « l'attribut $a$ ».

Un attribut correspond à une dimension de l'observation, à un « aspect » d'un fait.

C'est-à-dire que le contexte permet de déterminer de façon univoque et non ambigüe le type associé à l'identifiant au sein de l'attribut.

© 2026, CoFELI [CC BY-NC-SA 4.0]

La mise en page des expressions formelles des trois prochaines sous-sections (tuple, relation, base) varie d'une sous-section à l'autre. C'est volontaire. Nous y expérimentons trois mécanismes différents mis à disposition par AsciiDoc pour ce faire.

Nous désirons recueillir ainsi les commentaires de nos lecteurs, afin de choisir ce qui convient le mieux. Vos commentaires sont donc bienvenus et attendus !

© 2026, CoFELI [CC BY-NC-SA 4.0]

## 2.4. Tuple

Un tuple, en tant qu'objet de la théorie relationnelle, est une « composition » d'attributs.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble d'attributs.

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $n$ un entier naturel,
soit $n$ identifiants **distincts** dénotés $a_i \mid 1 \leq i \leq n$,
soit $n$ types (pas nécessairement distincts) dénotés $D_i \mid 1 \leq i \leq n$,
soit $n$ valeurs (pas nécessairement distinctes) dénotées $v_i \mid (a_i \in D_i) \wedge (1 \leq i \leq n)$
alors
un tuple $t$ est défini comme suit :

$$t \triangleq (\{a_i{:}D_i \mid 1 \leq i \leq n\}; \{(a_i,v_i) \mid 1 \leq i \leq n\})$$

ou, de façon équivalente, en extension :

$$t \triangleq (\{a_1{:}D_1, \ldots, a_i{:}D_i, \ldots, a_n{:}D_n\}; \{(a_1,v_1), \ldots, (a_i,v_i), \ldots (a_n,v_n)\})$$

avec

$$\forall\, i{:}1 \leq i \leq deg(t) \Rightarrow val(t,a_i) \in def(t,a_i)$$

© 2026, CoFELI [CC BY-NC-SA 4.0]

où

| | |
|---|---|
| $deg(t) = n$ | degré de t |
| $id(t) = \{a_i \mid 1 \leq i \leq n\}$ | ensemble des identifiants d'attributs de t |
| $def(t) = \{a_i{:}D_i \mid 1 \leq i \leq n\}$ | entête de t |
| $val(t) = \{(a_i,v_i) \mid 1 \leq i \leq n\}$ | valeur de t |
| $ANTE\ a \in id(t) :$ $def(t, a) = D \mid a{:}D \in def(t)$ | type de l'attribut $a_i$ de t |
| $ANTE\ a \in id(t) :$ $val(t, a) = v \mid (a,v) \in val(t)$ | valeur de l'attribut $a_i$ de t |

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Observation*

Pourquoi autoriser $deg(t) = 0$ ?

- Le cas $deg(t) = 0$ correspond à un tuple remarquable $(\{\}; \{\})$ souvent désigné par l'identifiant $t_0$.

*Exercice*

Pourquoi ce tuple est-il remarquable… et essentiel ?

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Autres dénotations fréquentes*

Parmi les plus fréquentes :

- dénotation des tuples fondée sur l'ordre d'énumération des attributs (les identifiants d'attributs et leurs types étant déterminés par ailleurs) :
  - $<v_1, v_2, \ldots, v_n>$
- dénotation des attributs au sein d'un tuple, *val(t, a)* :
  - *t.a*
  - *t[a]*
  - *a(t)*
  - *t(a)*

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 2.5. Relation

Une relation, en tant qu'objet de la théorie relationnelle, est un ensemble de tuples.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble de tuples de même type.

Une relation est la représentation d'un prédicat logique qui correspond à un ensemble d'observations de même « nature ».

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $a_i$ des identifiants distincts,
soit $D_j$ des types,
soit $t_j$ des tuples alors
une relation R est définie comme suit :

$$R \triangleq (\{a_1{:}D_1, \ldots, a_i{:}D_i, \ldots, a_n{:}D_n\}; \{t_1, \ldots, t_k, \ldots, t_m\})$$

avec

$$\forall\ 1 \leq k \leq card(R) : def(R) = def(t_k)$$

© 2026, CoFELI [CC BY-NC-SA 4.0]

où

| | |
|---|---|
| $deg(R) = n$ | le degré de R |
| $card(R) = m$ | la cardinalité de R |
| $id(R) = \{a_1, \ldots, a_i, \ldots, a_n\}$ | l'ensemble des identifiants d'attributs de R |
| $val(R) = \{t_1, \ldots, t_k, \ldots, t_m\}$ | la valeur de R |
| $def(R) = \{a_1{:}D_1, \ldots, a_i{:}D_i, \ldots, a_n{:}D_n\}$ | l'entête de R |
| $def(R, a_i) = D_i$ | le type de l'attribut $a_i$ de R |

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Illustration*

# Construction de la représentation compacte

© 2026, CoFELI [CC BY-NC-SA 4.0]

Quatre tuples (ayant le même entête)

| | | | |
|---|---|---|---|
| matricule: Matricule<br>matricule: 15113150 | nom: Nom<br>nom: Paul | adresse: Ville<br>adresse: >ᕕᖅᓂᑐᖅ | t1 |
| matricule: Matricule<br>matricule: 15112354 | nom: Nom<br>nom: Éliane | adresse: Ville<br>adresse: Blanc-Sablon | t2 |
| matricule: Matricule<br>matricule: 15113870 | nom: Nom<br>nom: Mohamed | adresse: Ville<br>adresse: Tadoussac | t3 |
| matricule: Matricule<br>matricule: 15110132 | nom: Nom<br>nom: Sergeï | adresse: Ville<br>adresse: Chandler | t4 |

Une relation comprenant quatre tuples

| matricule: Matricule | nom: Nom | adresse: Ville | |
|---|---|---|---|
| matricule: Matricule<br>matricule: 15113150 | nom: Nom<br>nom: Paul | adresse: Ville<br>adresse: >ᕕᖅᓂᑐᖅ | t1 |
| matricule: Matricule<br>matricule: 15112354 | nom: Nom<br>nom: Éliane | adresse: Ville<br>adresse: Blanc-Sablon | t2 |
| matricule: Matricule<br>matricule: 15113870 | nom: Nom<br>nom: Mohamed | adresse: Ville<br>adresse: Tadoussac | t3 |
| matricule: Matricule<br>matricule: 15110132 | nom: Nom<br>nom: Sergeï | adresse: Ville<br>adresse: Chandler | t4 |

La représentation compacte usuelle de cette même relation

| matricule: Matricule | nom: Nom | adresse: Ville |
|---|---|---|
| 15113150 | Paul | >ᕕᖅᓂᑐᖅ |
| 15112354 | Éliane | Blanc-Sablon |
| 15113870 | Mohamed | Tadoussac |
| 15110132 | Sergeï | Chandler |

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Prédicat et relation*

Un prédicat peut être défini de deux façons et, corolairement, une relation aussi :

- par énumération (l'ensemble de tous les énoncés vrais et eux seuls) ;
- par compréhension (la caractérisation nécessaire et suffisante des relations entre les variables).

## *Observation*

Pourquoi autoriser $deg(R) = 0$ ?

Le cas $deg(R) = 0$ correspond à deux (valeurs de) relations remarquables souvent désignées par $R_\varnothing$ et $R_0$ :

- $R_\varnothing = (\{\}; \{\})$
- $R_0 = (\{\}; t_0)$

Elles sont très importantes, comme le zéro et le un pour les entiers !

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Dénotation des constructeurs de types et de valeurs*

Il est important, en pratique et du point de vue du génie logiciel, de différencier syntaxiquement le constructeur de types du constructeur de valeurs, ce que Tutorial D ne fait pas de façon assez explicite, nous semble-t-il.

Attendu la solution retenue par Discipulus (voir section précédente), les (deux seules) valeurs de relations sans attributs sont représentées par

- `RELATION []` et
- `RELATION [TUPLE []]`.

Cela ne rappelle-t-il pas quelque chose aux Bourbakistes parmi vous ?

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 2.6. Base de données

Une base de données, en tant qu'objet de la théorie relationnelle, est une « composition » de relations.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble de relations.

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $v_i$ des identifiants distincts,
soit $D_i$ des types de relation,
soit $r_i$ des (valeurs de) relations alors
une base B est définie comme suit :

$$B \triangleq (\{v_1{:}D_1, \ldots, v_i{:}D_i, \ldots, v_n{:}D_n\}; \{r_1, \ldots, r_i, \ldots, r_n\})$$

avec

$$\forall\ 1 \leq i \leq card(B) : def(B, v_i) = def(r_i)$$

© 2026, CoFELI [CC BY-NC-SA 4.0]

avec

| | |
|---|---|
| $deg(B) = n$ | degré de B |
| $id(B) = \{v_1, \ldots, v_i, \ldots, v_n\}$ | ensemble des identifiants de B |
| $val(B) = \{r_1, \ldots, r_i, \ldots, r_n\}$ | valeur de B |
| $def(B) = \{v_1{:}D_1, \ldots, v_i{:}D_i, \ldots, v_n{:}D_n\}$ | entête de B |
| $def(B, a_i) = D_i$ | type de $a_i$ de B |

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3. Opérateurs relationnels

L'algèbre relationnelle est souvent présentée à l'aide de six opérateurs relationnels primaires :

- **renommage**, $R\ \rho\ a{:}b$, la relation comprenant tous les tuples formés à partir d'un tuple de $R$ dont l'attribut de nom $a$ est remplacé par un attribut de nom $b$ de même valeur, et rien d'autre ;
- **restriction**, $R\ \sigma\ c$, la relation comprenant tous les tuples de $R$ satisfaisant la condition $c$, et rien d'autre ;
- **projection**, $R\ \pi\ x$, la relation comprenant tous les tuples formés à partir d'un tuple de R dont seuls les attributs dont le nom est parmi $x$ ont été conservés, et rien d'autre ;

© 2026, CoFELI [CC BY-NC-SA 4.0]

- **jointure**, $R \bowtie S$, la relation comprenant tous les tuples formés des attributs d'un tuple de $R$ et de ceux d'un tuple de $S$ dont les attributs de même nom sont de même valeur, et rien d'autre ;
- **union**, $R \cup S$, la relation comprenant tous les tuples de $R$ et tous les tuples de $S$, et rien d'autre ;
- **différence**, $R - S$, la relation comprenant tous les tuples de $R$ qui ne sont pas dans $S$, et rien d'autre.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1. Opérateurs primaires

Définition relativement à la théorie des ensembles et à la représentation précédente (au modèle, à la structure).

La présence de l'entête dans chacun des tuples et chacune des relations permet de définir une opération structurelle, le renommage.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.1. Renommage

**R ρ A:C**

| A | B |
|---|---|
| a1 | b1 |
| a2 | b2 |
| a3 | b3 |

**ρ A:C =**

| C | B |
|---|---|
| a1 | b1 |
| a2 | b2 |
| a3 | b3 |

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ une relation,

soit $a$ et $b$ deux identifiants alors

R ρ a:b ≡

`R RENAME {a AS b}` ≡

`select` $a_1$`, …, a as b, …,` $a_n$ `from R` ≡

$ANTE\ a \in id(R) \bigwedge b \notin id(R) :$

$SOIT\ E := def(R,a)\ ;\ Z := def(R) - \{a{:}E\} \cup \{b{:}E\} :$

$(Z\ ;\ \{(Z\ ;\ val(t) - \{(a,val(t,a))\} \cup \{(b,val(t,a))\}) \mid t \in val(R)\})$

*Corolaires*

- $card(R) = card(R\ \rho\ a{:}b)$
- $\forall\ t \in val(R) : (Z\ ;\ val(t) - \{(a,val(t,a))\} \cup \{(b,val(t,a)) \in R\ \rho\ a{:}b$
- $a \notin id(R\ \rho\ a{:}b) \bigwedge b \in id(R\ \rho\ a{:}b)$
- $R = ((R\ \rho\ a{:}b)\ \rho\ b{:}a)$

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.2. Projection

$R\ \pi\ \{A, C\}$

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a2 | b2 | c2 |
| a3 | b3 | c3 |

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $w$ une expression ayant pour valeur une liste d'identifiants d'attribut $\{w_0, \ldots, w_n\}$ alors

$R\ \pi\ w \equiv$

`R {w₀, …, wₙ}` $\equiv$

`select w₁, …, wₙ from R` $\equiv$

$ANTE\ w \subseteq id(R) :$

$\quad SOIT\ Z := \{a{:}D \mid a \in id\ (R) \bigwedge a \in w\} :$

$\quad\quad (Z ; \{(Z ; \{(a,v) \mid (a,v) \in val(t) \bigwedge a \in w\}) \mid t \in val(R)\})$

L'opération de projection (applicable tant aux tuples qu'aux relations) n'est bien définie que si tous les identifiants d'attributs sont définis dans l'entête de la relation.

*Note*

- En Tutorial D et en SQL, la liste doit être explicite.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.3. Restriction

**R σ cond**

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a2 | b2 | c2 |
| a3 | b3 | c3 |

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ une relation,
soit $c$, une condition (expression booléenne),
soit $id(c)$, l'ensemble des identifiants d'attribut utilisés par $c$ alors

R σ c ≡

`R WHERE c` ≡

`select * from R where c` ≡

$ANTE\ id(c) \subseteq id(R) :$

$(def(R); \{t \mid t \in val(R) \bigwedge c(t)\})$

L'opération de restriction n'est bien définie que si tous les identifiants d'attributs de la condition sont définis dans l'entête de la relation.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.4. Jointure

$R \bowtie S$

| A | B |
|---|---|
| a1 | b1 |
| a2 | b1 |
| a3 | b3 |
| a4 | b4 |

$\bowtie$

| B | C |
|---|---|
| b1 | c1 |
| b2 | c2 |
| b3 | c3 |
| b3 | c4 |

=

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a2 | b1 | c1 |
| a3 | b3 | c3 |
| a3 | b3 | c4 |

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ et $S$ deux relations alors

$R \bowtie S \equiv$

`R JOIN S` $\equiv$

`select * from R natural join S` $\equiv$

$$
\begin{array}{l}
SOIT\ X := id(R) \cap id(S) : \\
\quad ANTE\ \forall\ a \in X : a{:}D \in def(R) \bigwedge a{:}D \in def(S) : \\
\quad\quad SOIT\ Z := def(R) \cup def(S) : \\
\quad\quad\quad (Z ; \{(Z ; val(t1) \cup val(t2)) \mid \\
\quad\quad\quad\quad t1 \in val(R) \bigwedge t2 \in val(S) \bigwedge (\forall\ a \in X : (a,v) \in val(t1) \bigwedge (a,v) \in val(t2))\})
\end{array}
$$

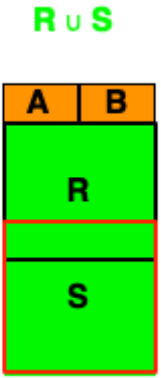
© 2026, CoFELI [CC BY-NC-SA 4.0]

*Note*

- La jointure naturelle n'est pas commutative en SQL en raison de l'ordonnancement des attributs dans un tuple, puisque le tuple y est défini comme une liste ordonnée d'attributs et non comme un ensemble d'attributs.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.5. Union




© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ et $S$ deux relations alors

$R \cup S \equiv$

`R UNION S` $\equiv$

`select * from R union select * from S` $\equiv$

$ANTE\ def(R) = def(S) :$

$(def(R) ; val(R) \cup val(S))$

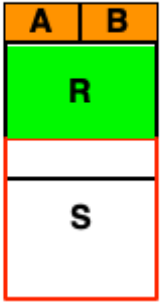
*Note*

- L'antécédent doit être aménagé en cas de relaxation de la compatibilité.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.6. Différence

R – S




© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ et $S$ deux relations alors

$\mathrm{R} - \mathrm{S} \equiv$

`R MINUS S` $\equiv$

`select * from R except select * from S` $\equiv$

$ANTE\ def(R) = def(S) :$

$(def(R); val(R) - val(S))$

*Note*

- L'antécédent doit être aménagé en cas de relaxation de la compatibilité.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.1.7. Synthèse des opérateurs relationnels primaires

**Restriction**

**R σ cond**

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a2 | b2 | c2 |
| a3 | b3 | c3 |

**Projection**

**R π {A, C}**

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a2 | b2 | c2 |
| a3 | b3 | c3 |

**Jointure**

**R ⋈ S**

| A | B |
|---|---|
| a1 | b1 |
| a2 | b1 |
| a3 | b3 |
| a4 | b4 |

⋈

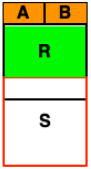
| B | C |
|---|---|
| b1 | c1 |
| b2 | c2 |
| b3 | c3 |
| b3 | c4 |

=

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a2 | b1 | c1 |
| a3 | b3 | c3 |
| a3 | b3 | c4 |

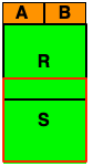
**Différence**

**R – S**




**Union**

**R ∪ S**




**Renommage**

**R ρ A:C**

| A | B |
|---|---|
| a1 | b1 |
| a2 | b2 |
| a3 | b3 |

ρ A:C =

| C | B |
|---|---|
| a1 | b1 |
| a2 | b2 |
| a3 | b3 |

© 2026, CoFELI [CC BY-NC-SA 4.0]

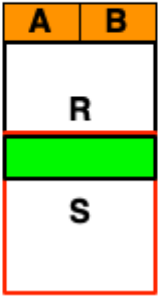
## 3.2. Opérateurs usuels

Afin d'augmenter l'expressivité de l'algèbre relationnelle, il est possible de définir d'autres opérateurs en terme des opérateurs relationnels primaires. En voici quelques-uns.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.2.1. Intersection

$$R \cap S = R - (R-S) = R \bowtie S$$




© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ et $S$ deux relations :

$R \cap S \equiv$

`R INTERSECT S` $\equiv$

`select * from R intersect select * from S` $\equiv$

*ANTE def(R) = def(S) :*

$R \bowtie S$

*Notes*

- L'antécédent garantit que $R \bowtie S = R - (R - S)$.
- L'antécédent doit être aménagé en cas de relaxation de la compatibilité.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.2.2. Produit (cartésien)

$$R \times S = R \bowtie S$$

| A |
|---|
| x |
| y |

$\times$

| B |
|---|
| a |
| b |
| c |

$=$

| A | B |
|---|---|
| x | a |
| x | b |
| x | c |
| y | a |
| y | b |
| y | c |

© 2026, CoFELI [CC BY-NC-SA 4.0]

Soit $R$ et $S$ deux relations alors

$\mathrm{R} \times \mathrm{S} \equiv$

`R TIMES S` $\equiv$

`select * from R cross join S` $\equiv$

$ANTE\ id(R) \cap id(S) = \emptyset :$

$R \bowtie S$

*Notes*

- L'antécédent garantit que $R \bowtie S = R \times S$.
- L'antécédent doit être aménagé en cas de relaxation de la compatibilité.
- Le produit cartésien n'est pas commutatif en SQL.

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.2.3. Semi-jointure (*matching*)

$$R \ltimes S = (R \bowtie S)\ \pi\ R$$

| A | B |
|---|---|
| a1 | b1 |
| a2 | b1 |
| a3 | b3 |
| a4 | b4 |

$\ltimes$

| B | C |
|---|---|
| b1 | c1 |
| b2 | c2 |
| b3 | c3 |
| b3 | c4 |

=

| A | B |
|---|---|
| a1 | b1 |
| a2 | b1 |
| a3 | b3 |

Soit $R$ et $S$ deux relations alors

$R \ltimes S \equiv$

`R MATCHING S` $\equiv$

`select` $r_1$, …, $r_n$ `from R natural join S` $\equiv$

$(R \bowtie S)\ \pi\ id(R)$

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.2.4. Semi-différence (*not matching*)

$$R \ltimes S = R - (R \ltimes S)$$

| A | B |
|---|---|
| a1 | b1 |
| a2 | b1 |
| a3 | b3 |
| a4 | b4 |

$\ltimes$

| B | C |
|---|---|
| b1 | c1 |
| b2 | c2 |
| b3 | c3 |
| b3 | c4 |

=

| A | B |
|---|---|
| a4 | b4 |

Soit $R$ et $S$ deux relations alors

$R \ltimes S \equiv$

`R NOT MATCHING S` $\equiv$

`select * from R except select` $r_1$, …, $r_n$ `from R natural join S` $\equiv$

$R - (R \ltimes S)$

© 2026, CoFELI [CC BY-NC-SA 4.0]

# 3.2.5. Extension

$$R\ \xi\ \{C{:=}f(A,B)\} = (R \bowtie F)\ \rho\ f'{:}C$$

| A | B |
|---|---|
| a1 | b1 |
| a2 | b1 |
| a3 | b3 |
| a4 | b4 |

$\xi\ \{C{:=}f(A,B)\} =$

| A | B | C |
|---|---|---|
| a1 | b1 | f(a1,b1) |
| a2 | b1 | f(a2,b1) |
| a3 | b3 | f(a3,b3) |
| a4 | b4 | f(a4,b4) |

*Définition*

Une relation $F$ correspond à une fonction $f(p_1{:}T_1, \ldots, p_n{:}T_1)$ de type $T$ lorsque

$def(F) = \{p_1{:}T_1, \ldots, p_n{:}T_1, f{:}T\} \bigwedge F = (F\ \sigma\ f{=}f(p_1, \ldots, p_n))$

© 2026, CoFELI [CC BY-NC-SA 4.0]

## 3.2.5.1. Solution utilisant les seuls opérateurs relationnels primaires

Soit $R$ une relation,

soit $F$ la relation correspondant à la fonction $f(p_1{:}T_1, \ldots, p_n{:}T_1)$ de type $T$ alors

$R\ \xi\ \{c := f(p_1, \ldots, p_n)\} \equiv$

`R EXTEND {c := f(p1, …, pn)}` $\equiv$

`select R.*, f(p1, …, pn) as c from R` $\equiv$

*ANTE*

$c \notin id(R) \bigwedge$

$def(R) \supseteq def(F) - \{f{:}T\} \bigwedge$

$R\ \pi\ \{p_1, \ldots, p_n\} \subseteq F\ \pi\ \{p_1, \ldots, p_n\}$ :

$R \bowtie (F\ \rho\ f{:}c)$

© 2026, CoFELI [CC BY-NC-SA 4.0]

## *Notes*

- L'antécédent signifie simplement que
  (1) l'identifiant $c$ ne dénote pas déjà un attribut de $R$,
  (2) chaque paramètre de $F$ correspond à un attribut de $R$,
  (3) tout tuple de $R$ à son correspondant dans $F$.
- La construction de relations en lieu et place des fonctions est un élément important dans la démonstration de la complétude de l'algèbre relationnelle.
- Le symbole utilisé pour l'extension varie beaucoup d'un auteur à l'autre. Certains utilisent $\alpha$. D'autres présentant l'opération comme une extension de la projection et utilisent conséquemment le symbole $\pi$.

© 2026, CoFELI [CC BY-NC-SA 4.0]

*Exercice*

- En pratique, plusieurs affectations peuvent être spécifiées dans la portée de l'extension, elles sont réputées être réalisées indépendamment il est donc possible de les exécuter concurremment.
  - En conséquence, $R \xi \{r_1 := exp_1 ; r_2 := exp_2\}$ pourrait ne pas être équivalent à $(R \xi \{r_1 := exp_1\}) \xi \{r_2 := exp_2\}$ en particulier si $exp_2$ fait appel à l'attribut $r_1$.
- Qu'en est-il en **SQL** ? Par exemple, quel est le résultat de
  - ```
    select 5 as x, x as y
    from (values (1, 2)) as A (x, y) ;
    ```

© 2026, CoFELI [CC BY-NC-SA 4.0]
    ```

## 3.2.5.2. Solution utilisant le modèle

Soit $R$ une relation,

soit $f(p_1{:}T_1, \ldots, p_n{:}T_1)$ une fonction de type $T$ alors

$R\ \xi\ \{c := f(p_1, \ldots, p_n)\} \equiv$

`R EXTEND {c := f(p1, …, pn)}` $\equiv$

`select *, f(p1, …, pn) as c from R` $\equiv$

$$
\begin{array}{l}
ANTE \\
\quad c \notin id(R) \bigwedge \\
\quad \{p_1, \ldots, p_n\} \subseteq id(R) : \\
\quad\quad SOIT \\
\quad\quad\quad Z := def(R) \cup \{c{:}T\} : \\
\quad\quad\quad\quad (Z\,; \{(Z\,; val(t) \cup \{(c, f(t.p_1, \ldots, t.p_n))\}) \mid t \in val(R)\})
\end{array}
$$

© 2026, CoFELI [CC BY-NC-SA 4.0]

Produit le 2026-04-17 15:07:06 UTC

**Institut catholique des arts et métiers**
**Université catholique d'Afrique centrale - Université Loyola du Congo**

© 2026, CoFELI [CC BY-NC-SA 4.0]